

Группа: M3206

К работе допущен:

Студенты: Абубакиров Аскар

Работа выполнена:

Преподаватель: Гумбатов Владислав
Юрьевич

Отчет принят:

Рабочий протокол и отчет по лабораторной работе № 6

Работа с модулем FUSE

Конвертация файлов на лету

```
#define FUSE_USE_VERSION 31

#include <fuse3/fuse.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <limits.h>

char *root_dir = NULL;

int has_suffix(const char *s, const char *suffix) {
    size_t ls = strlen(s), lf = strlen(suffix);
    return ls >= lf && strcmp(s + ls - lf, suffix) == 0;
}

void full_path(char out[PATH_MAX], const char *path) {
    snprintf(out, PATH_MAX, "%s%s", root_dir, path);
}

int jpg_to_png_path(char out[PATH_MAX], const char *path) {
```

```

    if (!has_suffix(path, ".jpg")) return 0;

    char tmp[PATH_MAX];
    snprintf(tmp, PATH_MAX, "%s", path);

    size_t len = strlen(tmp);
    tmp[len - 4] = '\0';
    snprintf(out, PATH_MAX, "%s%s.png", root_dir, tmp);
    return 1;
}

int convert_png_to_temp_jpg(const char *png_path, char jpg_path[PATH_MAX]) {
    char template[] = "/tmp/png2jpgfs_XXXXXX.jpg";
    int fd = mkstemp(template, 4);
    if (fd == -1) return -errno;
    close(fd);

    snprintf(jpg_path, PATH_MAX, "%s", template);

    if (access("/usr/bin/convert", X_OK) != 0) {
        fprintf(stderr, "Error: 'convert' not found\n");
        unlink(jpg_path);
        return -ENOENT;
    }

    pid_t pid = fork();
    if (pid == -1) {
        unlink(jpg_path);
        return -errno;
    }

    if (pid == 0) {
        execlp("convert", "convert", png_path, jpg_path, (char *)NULL);
        perror("execlp");
        _exit(127);
    }

    int status = 0;
    if (waitpid(pid, &status, 0) == -1) {
        unlink(jpg_path);
        return -errno;
    }

    if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
        fprintf(stderr, "Error converting %s to %s\n", png_path, jpg_path);
        unlink(jpg_path);
        return -EIO;
    }

    return 0;
}

int fs_getattr(const char *path, struct stat *st, struct fuse_file_info *fi) {
    (void) fi;
    memset(st, 0, sizeof(struct stat));
}

```

```

char real[PATH_MAX];
full_path(real, path);

if (lstat(real, st) == 0) return 0;

char png[PATH_MAX];
if (jpg_to_png_path(png, path) && lstat(png, st) == 0 && S_ISREG(st->st_mode)) {
    char tmp_jpg[PATH_MAX];
    int rc = convert_png_to_temp_jpg(png, tmp_jpg);
    if (rc != 0) return rc;

    int res = lstat(tmp_jpg, st);
    unlink(tmp_jpg);
    if (res == -1) return -errno;

    st->st_mode = S_IFREG | 0444;
    return 0;
}

return -errno;
}

int fs_readdir(const char *path, void *buf, fuse_fill_dir_t filler,
               off_t offset, struct fuse_file_info *fi,
               enum fuse_readdir_flags flags) {
    (void) offset;
    (void) fi;
    (void) flags;

    char real[PATH_MAX];
    full_path(real, path);

    DIR *dp = opendir(real);
    if (!dp) return -errno;

    filler(buf, ".", NULL, 0, 0);
    filler(buf, "..", NULL, 0, 0);

    struct dirent *de;
    while ((de = readdir(dp)) != NULL) {
        if (strcmp(de->d_name, ".") == 0 || strcmp(de->d_name, "..") == 0)
            continue;

        filler(buf, de->d_name, NULL, 0, 0);

        if (has_suffix(de->d_name, ".png")) {
            char jpg_name[PATH_MAX];
            snprintf(jpg_name, PATH_MAX, "%s", de->d_name);
            jpg_name[strlen(jpg_name) - 4] = '\0';
            strncat(jpg_name, ".jpg", PATH_MAX - strlen(jpg_name) - 1);
            filler(buf, jpg_name, NULL, 0, 0);
        }
    }

    closedir(dp);
    return 0;
}

```

```

}

int fs_open(const char *path, struct fuse_file_info *fi) {
    char real[PATH_MAX];
    full_path(real, path);

    int fd = open(real, fi->flags);
    if (fd != -1) {
        close(fd);
        return 0;
    }

    char png[PATH_MAX];
    if (jpg_to_png_path(png, path)) {
        if (access(png, R_OK) == 0) return 0;
    }

    return -errno;
}

int fs_read(const char *path, char *buf, size_t size, off_t offset,
            struct fuse_file_info *fi) {
    (void) fi;

    char real[PATH_MAX];
    char tmp_jpg[PATH_MAX];
    int is_virtual_jpg = 0;

    char png[PATH_MAX];
    if (jpg_to_png_path(png, path) && access(png, R_OK) == 0) {
        int rc = convert_png_to_temp_jpg(png, tmp_jpg);
        if (rc != 0) return rc;
        snprintf(real, PATH_MAX, "%s", tmp_jpg);
        is_virtual_jpg = 1;
    } else {
        full_path(real, path);
    }

    int fd = open(real, O_RDONLY);
    if (fd == -1) {
        if (is_virtual_jpg) unlink(tmp_jpg);
        return -errno;
    }

    int res = pread(fd, buf, size, offset);
    if (res == -1) res = -errno;

    close(fd);
    if (is_virtual_jpg) unlink(tmp_jpg);

    return res;
}

int fs_mkdir(const char *path, mode_t mode) {
    char real[PATH_MAX];
    full_path(real, path);

```

```

    int res = mkdir(real, mode);
    return res == -1 ? -errno : 0;
}

int fs_unlink(const char *path) {
    char real[PATH_MAX];
    full_path(real, path);
    int res = unlink(real);
    return res == -1 ? -errno : 0;
}

int fs_rmdir(const char *path) {
    char real[PATH_MAX];
    full_path(real, path);
    int res = rmdir(real);
    return res == -1 ? -errno : 0;
}

int fs_rename(const char *from, const char *to, unsigned int flags) {
    if (flags) return -EINVAL;

    char real_from[PATH_MAX];
    char real_to[PATH_MAX];
    full_path(real_from, from);
    full_path(real_to, to);

    int res = rename(real_from, real_to);
    return res == -1 ? -errno : 0;
}

int fs_create(const char *path, mode_t mode, struct fuse_file_info *fi) {
    char real[PATH_MAX];
    full_path(real, path);

    int fd = open(real, fi->flags, mode);
    if (fd == -1) return -errno;

    close(fd);
    return 0;
}

struct fuse_operations operations = {
    .getattr = fs_getattr,
    .readdir = fs_readdir,
    .open = fs_open,
    .read = fs_read,
    .mkdir = fs_mkdir,
    .unlink = fs_unlink,
    .rmdir = fs_rmdir,
    .rename = fs_rename,
    .create = fs_create,
};

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Usage: %s <original_dir> <mountpoint> [FUSE options]\n",

```

```

argv[0]);
    return 1;
}

root_dir = realpath(argv[1], NULL);

if (!root_dir) {
    perror("realpath");
    return 1;
}

int fuse_argc = argc - 1;
char **fuse_argv = malloc(sizeof(char *) * fuse_argc);

if (!fuse_argv) {
    perror("malloc");
    free(root_dir);
    return 1;
}

fuse_argv[0] = argv[0];

for (size_t i = 2; i < argc; i++) {
    fuse_argv[i - 1] = argv[i];
}

int ret = fuse_main(fuse_argc, fuse_argv, &operations, NULL);

free(fuse_argv);
free(root_dir);

return ret;
}

```

```

[root@localhost lab6]# ls -lR
.:
total 40
-rwxr-xr-x. 1 root root 31512 May  7 12:21 file_conversion
-rw-r--r--. 1 root root 6664 May  7 14:47 file_conversion.c
drwxr-xr-x. 2 root root   6 May  7 15:36 mounted_dir
drwxr-xr-x. 2 root root  44 May  7 12:20 original_dir

./mounted_dir:
total 0

./original_dir:
total 8
-rw-r--r--. 1 root root 171 May  7 12:20 sample1.png
-rw-r--r--. 1 root root 177 May  7 12:20 sample2.png
[root@localhost lab6]# ./file_conversion original_dir mounted_dir
[root@localhost lab6]# ls plR
ls: cannot access 'plR': No such file or directory
[root@localhost lab6]# ls -lR
.:
total 40
-rwxr-xr-x. 1 root root 31512 May  7 12:21 file_conversion
-rw-r--r--. 1 root root 6664 May  7 14:47 file_conversion.c
drwxr-xr-x. 2 root root  44 May  7 12:20 mounted_dir
drwxr-xr-x. 2 root root  44 May  7 12:20 original_dir

./mounted_dir:
total 16
-r--r--r--. 1 root root 415 May  7 15:36 sample1.jpg
-rw-r--r--. 1 root root 171 May  7 12:20 sample1.png
-r--r--r--. 1 root root 559 May  7 15:36 sample2.jpg
-rw-r--r--. 1 root root 177 May  7 12:20 sample2.png

./original_dir:
total 8
-rw-r--r--. 1 root root 171 May  7 12:20 sample1.png
-rw-r--r--. 1 root root 177 May  7 12:20 sample2.png
[root@localhost lab6]# file mounted_dir/sample1.jpg
mounted_dir/sample1.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment length 16, baseline, precision 8, 100x100, frames 3
[root@localhost lab6]# _

```